



논문 리뷰: Efficient Memory Management for Large Language Model Serving with PagedAttention (vLLM)

0. 논문 정보

- **논문 제목:** Efficient Memory Management for Large Language Model Serving with PagedAttention
- **저자:** Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica
- **소속:** UC Berkeley, Stanford University, UC San Diego
- **발표:** SOSP 2023 (October 23–26, 2023, Koblenz, Germany)
- **arXiv:** [2309.06180](https://arxiv.org/abs/2309.06180)
- **코드:** github.com/vllm-project/vllm

[!note]

이 논문은 현재 LLM 서빙 시스템의 사실상 표준이 된 **vLLM**의 기반이 된 시스템 논문이다. OS의 가상 메모리 페이징 기법을 LLM의 KV 캐시 관리에 적용해 throughput을 기존 SOTA 대비 2~4배 향상시켰다.

논문이 다루는 분야

1. Introduction

논문이 다루는 분야

LLM 서빙 시스템(LLM Serving System) 의 메모리 효율성 및 throughput 최적화 분야이다. 즉, 학습된 LLM을 실제 서비스로 제공할 때 GPU 메모리를 어떻게 효율적으로 관리하여 단위 시간당 더 많은 요청(request)을 처리할 수 있는가에 대한 시스템 레벨의 연구이다.

해당 task에서 기존 연구 한계점

LLM 서빙에서 throughput을 높이려면 충분히 많은 요청을 **배치(batching)** 해서 처리해야 한다. 그런데 GPU 메모리에서 약 30%를 차지하는 **KV 캐시(Key-Value Cache)** 가 동적으로 증가/감소하기 때문에 효율적 관리가 매우 어렵다.

구체적인 수치로 보자면, 13B 파라미터 OPT 모델의 경우:

- 토큰 1개당 KV 캐시 크기 = $2 \text{ (K, V)} \times 5120 \text{ (hidden)} \times 40 \text{ (layers)} \times 2 \text{ bytes (FP16)}$
= **800 KB**
- 최대 시퀀스 길이 2048까지 가면 요청 1개당 **약 1.6 GB**가 필요
- A100 40GB 기준, 40GB 중 26GB는 모델 weight 차지 → KV 캐시로 쓸 수 있는 공간은 약 12GB뿐

기존 LLM 서빙 시스템(FasterTransformer, Orca)은 딥러닝 프레임워크가 텐서를 **연속 메모리(contiguous memory)** 에 저장하도록 요구하기 때문에, 각 요청에 대해 **최대 시퀀스 길이만큼의 연속된 메모리 청크를 미리 할당**한다. 이로 인해 세 가지 종류의 메모리 낭비가 발생한다.

1. **Reserved (예약 슬롯)**: 아직 생성되지 않은 미래 토큰을 위해 잡아둔 공간
2. **Internal Fragmentation (내부 단편화)**: 실제 출력 길이가 최대 길이보다 짧을 때 남는 공간 (예: 2048 예약했는데 100 토큰만 생성)
3. **External Fragmentation (외부 단편화)**: buddy allocator가 메모리를 할당할 때 청크 크기가 다양해서 생기는 빈틈

논문의 Figure 2에 따르면, 기존 Orca 시스템은 KV 캐시 메모리의 **20.4% ~ 38.2%만 실제로 사용**되고 나머지는 모두 낭비된다. 게다가 메모리가 연속적이어야 하므로 **parallel sampling**이나 **beam search**처럼 여러 시퀀스 간에 KV 캐시를 공유할 기회조차 활용할 수 없다.

논문의 contributions

저자들은 **PagedAttention** 이라는 새로운 attention 알고리즘을 제안한다. 이는 OS의 고전적인 **가상 메모리(virtual memory)** 와 **페이징(paging)** 아이디어에서 영감을 받았다. KV 캐시를 고정 크기의 **블록(block)** 으로 분할하고, 각 블록은 물리 GPU 메모리에서 **비연속적으로(non-contiguously)** 저장될 수 있다. 이 위에 구축한 LLM 서빙 엔진인 **vLLM** 은 다음 두 가지를 달성한다.

1. 거의 0에 가까운 KV 캐시 메모리 낭비 (논문 측정 96.3% 실효 사용률)
2. 요청 내부 및 요청 간 KV 캐시의 유연한 공유 (parallel sampling, beam search, shared prefix)

2. Related Work

2.1 Transformer 기반 LLM과 KV 캐시

LLM은 자기회귀(autoregressive) 방식으로 토큰을 하나씩 생성한다.

$$P(x) = P(x_1) \cdot P(x_2|x_1) \cdots P(x_n|x_1, \dots, x_{n-1})$$

self-attention 연산에서 각 위치 i 의 query, key, value 벡터는 다음과 같다.

$$q_i = W_q x_i, \quad k_i = W_k x_i, \quad v_i = W_v x_i$$

attention score와 출력은:

$$a_{ij} = \frac{\exp(q_i^\top k_j / \sqrt{d})}{\sum_{t=1}^i \exp(q_i^\top k_t / \sqrt{d})}, \quad o_i = \sum_{j=1}^i a_{ij} v_j$$

LLM 추론은 두 단계로 나뉜다.

- **Prompt phase (Prefill):** 입력 프롬프트 전체를 한꺼번에 처리하여 KV 캐시를 생성. 행렬-행렬 곱셈으로 GPU 병렬성을 잘 활용 → compute-bound
- **Autoregressive generation phase (Decode):** 토큰을 하나씩 순차적으로 생성. 이전 토큰들의 KV 캐시를 재사용. 행렬-벡터 곱셈이라 GPU를 충분히 활용 못 함 → **memory-bound** (서빙 latency의 대부분 차지)

2.2 기존 배칭 기법

- **Static batching:** 요청들이 다른 시간에 도착하고 길이가 다 다르므로 비효율적 (패딩 낭비)
- **Iteration-level scheduling (Orca):** 매 iteration마다 끝난 요청은 빼고 새 요청을 넣음. 패딩 불필요. 하지만 메모리 관리 문제는 여전히 해결 못 함

3. 제안 방법론

Main Idea

> OS 가상 메모리를 Attention에 가져오다

이 논문의 가장 통찰적인 부분은 "**KV 캐시 관리 문제 = OS 메모리 관리 문제**" 라는 매핑이다.

OS 가상 메모리	PagedAttention / vLLM
Page (페이지)	KV Block (고정 토큰 수의 K, V 벡터)
Byte	Token
Process	Request
Page Table	Block Table
Copy-on-Write	Beam search / Parallel sampling 공유
Swap to disk	Swap to CPU RAM
Shared library	Shared prefix (system prompt)

3.2 PagedAttention 알고리즘

KV 캐시를 고정 크기 블록(**B개의 토큰**) 으로 분할한다. 논문의 기본값은 **B = 16**이다.

- Key block: $K_j = (k_{(j-1)B+1}, \dots, k_{jB})$
- Value block: $V_j = (v_{(j-1)B+1}, \dots, v_{jB})$

Attention 연산은 블록 단위로 다음과 같이 변환된다.

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top$$

PagedAttention kernel은 attention 계산 시 block table을 참조해서 **물리 메모리 상 비연속적으로 흩어진 블록들**을 따로따로 가져와 query와 곱한다.

3.3 KV Cache Manager — Block Table

vLLM의 핵심 자료구조는 OS의 page table에 해당하는 **block table**이다.

- 각 요청의 KV 캐시는 **logical blocks**의 시퀀스로 표현됨 (좌 → 우로 채워짐)

- 각 logical block은 GPU 메모리의 **physical block**으로 매핑됨 (block table이 매핑 관리)
- 마지막 logical block은 미래 생성 토큰을 위해 일부만 채워져 있음
- Block table은 각 logical block에 대해 (1) 대응하는 physical block 번호, (2) 채워진 슬롯 수(#filled) 를 저장

결정적으로 **logical 블록은 연속이지만 physical 블록은 비연속이어도 된다**. 따라서 새 토큰이 생성될 때 미리 큰 청크를 잡아둘 필요가 없고, 필요할 때마다 빈 physical block을 하나씩 동적 할당한다.

메모리 낭비는 **요청당 마지막 블록의 빈 슬롯들로 제한된다**. B=16이면 요청당 최대 15 토큰 분량만 낭비 → 거의 0에 가까운 낭비.

Contribution

이 논문이 학계와 산업계에 기여한 바는 명확하다.

1. **LLM 서버의 메모리 할당 문제를 정량화**: 기존 시스템의 KV 캐시 메모리 효율을 20~38%로 측정하고, 이것이 throughput의 진짜 병목임을 보임
2. **PagedAttention 알고리즘 제안**: KV 캐시를 비연속 paged memory에 저장하는 attention 알고리즘
3. **vLLM 시스템 구현 및 오픈소스 공개**: PagedAttention 위에 구축한 분산 LLM 서버 엔진
4. **다양한 디코딩 시나리오 지원**: parallel sampling, beam search, shared prefix를 copy-on-write 메커니즘으로 효율적 처리
5. **2~4배 throughput 향상**: 동일 latency 기준 SOTA 대비 (긴 시퀀스, 큰 모델, 복잡한 디코딩일수록 개선폭 큼)

3.5 다양한 디코딩 시나리오에서의 적용

Parallel Sampling (병렬 샘플링)

한 요청이 여러 출력 시퀀스를 동시에 생성하는 경우 (예: GitHub Copilot이 여러 코드 후보 제시). 프롬프트 부분의 KV 캐시는 **모든 샘플이 공유**할 수 있다.

- 동일한 physical block에 여러 logical block이 매핑됨 → **reference count** 도입
- 샘플이 분기되어 새 토큰을 마지막 공유 블록에 쓰려고 하면 → **Copy-on-Write (COW)**: 새 physical block을 할당하고 기존 내용 복사 후 ref count 감소

- Alpaca 데이터셋 기준 **6.1% ~ 9.8% 메모리 절약**

Beam Search (빔 서치)

번역 등에서 top-k 후보를 유지하며 디코딩하는 알고리즘. OS의 process tree처럼 후보들이 분기/병합하며 **블록 공유 패턴이 동적으로 변한다**.

- 모든 빔 후보가 prompt 블록을 공유
- 후보들 간에 중간 블록도 광범위하게 공유 (Figure 9 참조)
- 빔 후보가 탈락하면 ref count가 0이 되어 자동으로 메모리 해제
- **기존 시스템: 후보 간 빈번한 메모리 복사 필요 → vLLM: 포인터(블록 ID)만 바꾸면 됨**
- 빔 폭 6일 때 **37.6% ~ 55.2% 메모리 절약**

Shared Prefix (공유 프리픽스)

System prompt나 few-shot 예시처럼 여러 요청이 동일한 프리픽스를 공유하는 경우. 이 프리픽스의 KV 캐시를 미리 계산해두면 prefill 연산까지 절약 가능. OS가 공유 라이브러리를 처리하는 방식과 동일.

3.6 스케줄링과 Preemption

요청 트래픽이 시스템 용량을 초과하면 일부 요청을 **선점(preempt)** 해야 한다. vLLM은 **FCFS(First-Come-First-Serve)** 정책을 쓰고, 두 가지 복구 메커니즘을 제공한다.

Swapping (스와핑)

- 선점된 시퀀스의 KV 블록을 **CPU RAM**으로 복사 (OS의 disk swap에 해당)
- 시퀀스가 재개되면 CPU → GPU로 다시 가져옴
- 큰 블록 사이즈에서 유리 (PCIe 대역폭 활용)

Recomputation (재계산)

- 선점된 시퀀스의 KV 캐시를 그냥 버림
- 재개될 때 원본 프롬프트 + 지금까지 생성된 토큰들을 새 프롬프트로 합쳐서 **한 번의 prefill로 다시 계산**
- 작은 블록 사이즈에서 유리 (PCIe 작은 전송이 비효율적)

- OS에는 없는 LLM 특유의 기법

3.7 Distributed Execution (분산 실행)

Megatron-LM 스타일 텐서 병렬화 지원. 핵심 관찰: tensor parallelism에서도 각 GPU worker는 같은 토큰들을 같은 위치에서 처리하므로 block table은 모든 worker에서 공유된다. → 중앙 집중식 KV cache manager 하나만 있으면 됨.

4. 실험 및 결과

4.1 실험 설정

- **모델:** OPT-13B, OPT-66B, OPT-175B, LLaMA-13B
- **GPU:** NVIDIA A100 40GB (175B는 8×A100-80GB)
- **데이터셋:** ShareGPT (긴 시퀀스, 변동성 큼), Alpaca (짧은 시퀀스)
- **요청 도착:** Poisson 분포로 다양한 request rate 시뮬레이션
- **비교 대상:**
 - FasterTransformer (latency 최적화 시스템)
 - Orca (Max): 최대 시퀀스 길이만큼 예약 (최악 케이스)
 - Orca (Pow2): 출력 길이의 최대 2배까지 예약
 - Orca (Oracle): 실제 출력 길이를 미리 안다고 가정 (Orca의 상한선, 현실에서는 불가능)
- **지표:** Normalized latency = 평균 end-to-end latency / 출력 길이

결과

Basic Sampling (단일 시퀀스 생성)

- ShareGPT 기준 vLLM은 Orca(Oracle) 대비 **1.7~2.7배**, Orca(Max) 대비 **2.7~8배** 더 높은 request rate를 동일 latency로 처리
- FasterTransformer 대비 **최대 22배** throughput 향상

- OPT-13B에서 vLLM은 Orca(Oracle)보다 **2.2배 더 많은 요청을 동시 배치**, Orca(Max)보다 **4.3배 더 많은 요청을 배치**

Parallel Sampling & Beam Search

- 샘플 수가 많아지거나 빔 폭이 커질수록 vLLM의 이득이 극대화됨
- 빔 폭 6, OPT-13B, Alpaca: vLLM이 Orca(Oracle) 대비 **2.3배** 향상 (basic sampling에서는 1.3배)
- 공유 가능성이 클수록 vLLM의 강점이 드러남

Shared Prefix (번역 워크로드)

LLaMA-13B, WMT16 영-독 번역 데이터셋:

- 1-shot prefix 공유: vLLM이 Orca(Oracle) 대비 **1.67배** throughput
- 5-shot prefix 공유: vLLM이 Orca(Oracle) 대비 **3.58배** throughput

Chatbot 워크로드

ShareGPT 데이터셋의 긴 대화 (대부분 1024 토큰 프롬프트): vLLM이 모든 Orca baseline 대비 **2배** request rate.

4.3 Ablation Studies

Kernel Microbenchmark

PagedAttention의 block table 참조 오버헤드: FasterTransformer 대비 attention kernel latency가 **20~26% 증가**. 하지만 attention 외 다른 연산(Linear 등)에는 영향 없어서 end-to-end로는 vLLM이 훨씬 빠르다.

Block Size의 영향

- 너무 작으면: GPU 병렬성 활용 부족
- 너무 크면: internal fragmentation 증가, 공유 가능성 감소
- **최적값: B=16** (vLLM 기본값으로 채택). ShareGPT는 16~128 모두 비슷, Alpaca(짧은 시퀀스)는 16~32가 최적

Recomputation vs Swapping

- 작은 블록($B < 16$): **Recomputation이 유리** (PCIe 작은 전송 비효율)
- 큰 블록($B > 64$): **Swapping이 유리** (PCIe 대역폭 활용)
- 중간 블록(16~64): 둘 다 비슷한 성능
- Recomputation 오버헤드는 swapping의 20%를 절대 넘지 않음

5. 결론 (배운점)

연구의 의의

1. 시스템 분야의 고전 기법(가상 메모리, 페이징, COW)을 ML 시스템에 우아하게 이식한 모범 사례이다. SOSP 2023에서 발표된 만큼 시스템 커뮤니티 관점에서 매우 깔끔한 분야 간 매핑이다.
2. LLM 서빙 시스템의 사실상 표준이 되어, 현재 거의 모든 대규모 LLM 추론 서비스(상용/오픈소스 모두)가 vLLM 혹은 그 derivative를 사용한다. 후속 연구(SGLang, LMDeploy, TensorRT-LLM 등)도 PagedAttention 아이디어를 기본 가정으로 채택.
3. 메모리 효율성 → throughput 향상이라는 명확한 인과 사슬을 정량적으로 입증. 단순히 "빠르다"가 아니라 "왜 빠른지"를 Figure 2의 메모리 활용률 비교로 보여줌.
4. 오픈소스로 공개되어 산업계 채택이 매우 빠르게 일어났다는 점에서 학계-산업계 임팩트 모두 큰 연구다.

한계점

1. **Attention kernel 오버헤드 (20~26%)**: block table 참조와 비연속 메모리 접근으로 인한 본질적 비용. 후속 연구에서 FlashAttention과의 결합(FlashAttention-2 + paged)으로 일부 해소되었지만 여전히 트레이드오프 존재.
2. 모든 GPU 워크로드에 적용되지는 않음: 저자들도 Discussion 섹션에서 명시했듯, 텐서 모양이 정적이고 compute-bound인 워크로드(예: DNN training, 비-LLM inference)에는 오히려 손해. 동적이고 memory-bound한 워크로드에만 효과적.
3. 블록 크기의 하이퍼파라미터 의존성: 워크로드 특성(시퀀스 길이 분포)에 따라 최적 블록 크기가 달라짐. $B=16$ 이 "평균적"으로 좋지만 자동 튜닝은 없음.
4. MQA/GQA, Quantization과의 상호작용 분석 부재: 논문 작성 시점에는 multi-head attention만 다뤘지만, 이후 GQA(Llama 2)나 KV cache quantization 같은 직교적인

메모리 절감 기법과의 시너지/충돌 분석은 후속 연구의 몫.

5. **공정성 문제**: FCFS 정책은 단순하지만, 짧은 요청이 긴 요청 뒤에서 대기하는 head-of-line blocking 문제가 있다. 후속 연구(Sarathi-Serve 등)가 이를 다루고 있다.

좋았던 점 (배운 점)

- **분야 간 매핑의 우아함**: "문제 X = 다른 분야의 잘 풀린 문제 Y"를 알아채는 통찰의 가치. 시스템 사람들에게는 너무 친숙한 페이지징을 ML 시스템에 가져온 것만으로 큰 임팩트.
- **Figure 2의 설득력**: 메모리 낭비를 reserved / internal frag / external frag로 분해하고, 각 baseline의 실제 사용률을 막대 그래프로 시각화한 단 한 장의 그림이 논문 전체의 문제 정의와 해결의 정당성을 모두 전달한다. 시스템 논문 figure의 모범 사례.
- **현실적인 디테일**: copy-on-write의 마지막 블록 처리, all-or-nothing eviction, recomputation 같은 LLM 특화 최적화가 OS 페이지징을 단순히 베끼는 게 아니라 **재해석**한 결과임을 보여준다.
- **오픈소스 + 페이퍼 동시 공개**의 영향력: 학계 페이퍼가 산업계 표준이 되는 가장 빠른 경로를 보여준 사례.

아쉬웠던 점

- 평가가 OPT 계열 중심이고 LLaMA는 prefix sharing 실험에서만 등장. 다양한 모델 아키텍처(특히 MoE)에서의 결과가 더 있었으면.
- Sequence length가 매우 긴 경우(128K+ context)의 동작 분석이 없음. 현재 long-context LLM 시대에는 이 부분이 더 중요해졌다.
- 분산 환경 실험이 단일 노드 내 tensor parallel에 한정됨. Multi-node serving의 KV 캐시 관리 이슈는 후속 연구로 남겨둬.

논문 리뷰 (Introduction): Adam: A Method for Stochastic Optimization

0. 논문 정보

- **논문 제목**: Adam: A Method for Stochastic Optimization

- **저자:** Diederik P. Kingma (University of Amsterdam, OpenAI), Jimmy Lei Ba (University of Toronto)
- **발표:** ICLR 2015 (최초 arXiv 업로드 2014년 12월 22일)
- **arXiv:** [1412.6980](https://arxiv.org/abs/1412.6980)

[!note]

이 노트는 **Introduction** 섹션만 정리한다. Adam은 현대 딥러닝의 사실상 기본 옵티마이저이며, 이 논문은 피인용 수 10만 회를 훨씬 넘긴 ML 역사상 가장 영향력 있는 논문 중 하나다.

1. Introduction

1.1 논문이 다루는 분야

- **확률적 경사하강법(stochastic gradient-based optimization)**의 개선 연구.
- 많은 문제는 **스칼라 목적 함수를 파라미터에 대해 최소화/최대화**하는 형태로 표현되며, 함수가 파라미터에 대해 미분 가능할 때 경사하강법이 일반적으로 효율적이다.
- **High-dimensional parameter space** (고차원 파라미터 공간) → Newton's method 같은 2차 최적화는 헤시안 계산이 불가능. **1차 방법론(first-order method)에 제한**해서 경쟁함.
- **목적 함수가 stochastic** — 데이터의 minibatch에 따라 평가되거나, dropout이 regularization으로 들어가는 경우 등.

1.2 기존 방법들의 강점과 한계

논문은 Introduction에서 **AdaGrad**와 **RMSProp**을 Adam의 직접적인 영감으로 명시적으로 언급한다.

방법	강점	약점
SGD	단순, 메모리 효율적	learning rate 수동 튜닝 필요, sparse gradient 취약, 모든 파라미터에 동일 lr
AdaGrad (Duchi et al., 2011)	sparse gradient에 강함, 파라미터별 adaptive lr	과거 gradient의 제곱합을 계속 누적 → 시간이 지남에 따라 lr이 과도하게 작

방법	강점	약점
		아저 학습 멈춤
RMSProp (Tieleman & Hinton, 2012)	non-stationary 목적, on-line 세팅에 강함 (gradient 제곱을 EMA로 관리)	momentum 결합 시 bias-correction 부재 → 초기 단계에서 불안정

이 둘의 장점을 한 알고리즘에 담아서, 동시에 초기화 bias까지 보정하고 싶다는 것이 Adam의 출발점이다.

1.3 제안하는 방법: Adam

Adam 은 **A**da ptive **M**oment estimation의 약자다. 1차 그래디언트만 사용하여, **그래디언트의 1차 모멘트(평균)과 2차 모멘트(분산 유사량)을 EMA로 추정해 파라미터별 adaptive learning rate를 계산한다.**

Introduction이 주장하는 Adam의 장점

1. 구현이 간단하다 — 몇 줄의 의사 코드로 표현 가능
2. 계산적으로 효율적 — 그래디언트 계산 이외의 오버헤드가 작음
3. 메모리 요구량이 적다 — 1차/2차 모멘트 벡터 두 개만 저장 (파라미터 수와 동일)
4. 그래디언트의 **diagonal rescaling에 불변** — 그래디언트를 상수배 스케일해도 update 크기 변하지 않음
5. 대규모 데이터/파라미터 문제에 적합
6. **non-stationary objective에 적합** — 목적 함수가 시간에 따라 변하는 상황 (온라인 학습, RL 등)
7. **noisy/sparse gradient에 강건**
8. 하이퍼파라미터가 직관적 해석을 가지며, 튜닝이 거의 필요 없다

더해서 Adam의 update rule 자체가 가지는 다음 성질도 Introduction에서 명시된다.

- **Update의 크기가 그래디언트 rescaling에 불변**
- **Stepsize가 하이퍼파라미터 α 에 의해 근사적으로 upper bound가 잡힌다** → trust region 느낌으로 α 튜닝이 직관적
- **stationary objective가 필요 없음**

- **sparse gradient**에서 잘 작동
- **자연스러운 step size annealing** — 최적점 근처에서 SNR(signal-to-noise ratio)가 0에 가까워져 step이 작아짐

1.4 Adam이 관련 알고리즘과 다른 점 (논문이 있제기하는 핵심)

Introduction은 Adam을 RMSProp·AdaGrad의 단순 합성이 아니라 **bias-correction**을 도입한 새로운 관점 으로 포지셔닝한다. 논문의 Algorithm 1은 다음과 같이 제시된다 (\$\alpha=0.001\$, \$\beta_1=0.9\$, \$\beta_2=0.999\$, \$\epsilon=10^{-8}\$ 기본값):

$$g_t = \nabla_{\theta} f_t(\theta_{t-1})$$

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\hat{m}_t = m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t) \quad (\text{bias-corrected})$$

$$\theta_t = \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

이 update rule의 핵심은:

- m_t, v_t 를 **영벡터로 초기화** 하면 초기에 제공된 계수 $1 - \beta^t$ 때문에 추정치가 0으로 치우친다 → **bias correction** 으로 이를 보정
- $\sqrt{\hat{v}_t}$ 로 나눠주는 것이 파라미터별 adaptive learning rate 역할
- RMSProp은 bias-correction이 없어서 $\beta_2 \rightarrow 1$ (sparse gradient 경우에 필요)에서 초기 step이 너무 커져서 divergence가 나기 쉬움